МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №1
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск с возвратом.

Студент гр. 4343 _____ Маркашов Г.Д.

Преподаватель _____ Курасов Н. С.

Санкт-Петербург
2026

Цель Работы.

Изучить принцип работы рекурсивного алгоритма поиска с возвратом и решить с его помощью задачу.

Задание.

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до N – 1 и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу – квадрат размера N. Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков см. на рисунке 1.

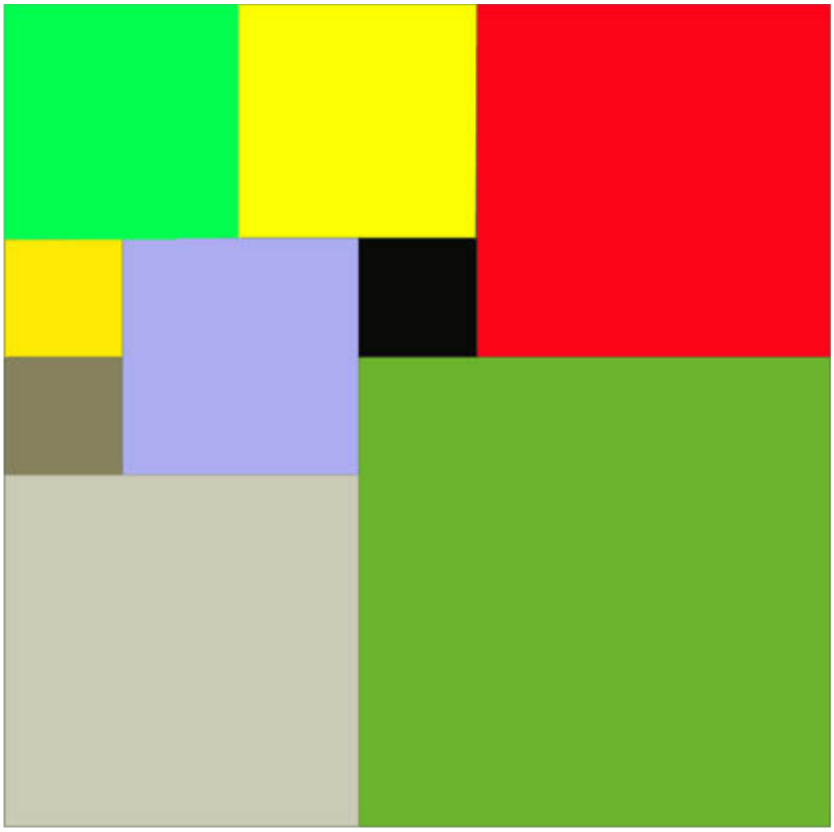

Рисунок 1. Пример построение квадрата N=7

Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Теоретический минимум

Метод поиска с возвратом (бэктрекинг) — это способ полного перебора вариантов путем последовательного расширения частичного решения. Если решение нельзя дополнить, алгоритм возвращается к предыдущему шагу и пробует другой путь. Так находятся все возможные решения.

Для ускорения процесса важно как можно раньше отсекать заведомо неподходящие варианты. Метод универсален и прост в реализации, но при росте данных время выполнения может стать неприемлемо большим.

Выполнение работы.

Для решения задачи был использован рекурсивный алгоритм поиска с возвратом на языке программирования C++. Основной алгоритм осуществляется с помощью перебора всех вариантов расположения квадратов от большего к меньшими, с помощью рекурсии, когда кол-во используемых квадратов было больше или равно минимальному ранее используемых алгоритм возвращался и искал более оптимальный способ. При достижении полного квадрата результат сравнивался с предыдущим и сохранялся при необходимости.

Перед запуском основного алгоритма идет проверка на четность числа N, в случае его четности используется заранее установленное наилучшее решение из 4х квадратов размера N/2.

Оптимизации.

Самый большой квадрат органичен размерами от N-1 до (N+1) / 2. Верхняя граница обусловлена ограничением самой задачи, а нижняя граница это минимальный размер квадрата установленного в угол, при которой оставшаяся область может быть разбита на меньшие квадраты без потери оптимального решения.

Для оптимизации было принято решение устанавливать самый большой квадрат только в левый верхний угол из-за чего алгоритм оптимизируется в десятки раз. При попытке смещения самого большого квадрата из угла приводит к образованию большого кол-ва пустых областей

минимального размера (1x1, 2x2, 3x3 и т.д.), что заранее не оптимальное решение.

Если на конфигурации на каком то из этапа выполняется равенство: текущее кол-во квадратов больше или равно минимальном, ранее найденному, кол-ву квадратом минус 1, при этом незаполненная площадь больше нуля, то алгоритм возвращается назад по стеку и пробует иные варианты расположения квадрата. Это работает из-за того, что если на следующем этапе будет поставлен квадрат и, даже, если площадь станет равна нулю, то решение будет точно таким же как уже найденное лучше. Таким образом оптимизация в несколько раз улучшает производительность алгоритма на больших N.

Одна из оптимизаций это способ хранения сетки квадрата. В текущей реализации используется одномерный массив размере N*N, а нужная клетка вычисляется по формуле $y * n + x$.

Сложность алгоритма.

Основной алгоритм является полным перебором с возвратом, он отсекает ветви рекурсии по лучшему решению, но по-прежнему является экспоненциальным по сложности O(c^n, где c примерно равняется 2). По памяти сложность O(n^2) это есть сетка квадрата.

Зависимость кол-ва операции от размера N.

Было замерено кол-во входа в рекурсию от числа N. Алгоритм для четных N всегда выдает результат равной 1 так как не используется рекурсия. Для всех остальных чисел зависимость оказалась c^n, где c примерно равняется 2. Эта константа зависит от факторизации самого числа N. Результат зависимости кол-ва операции от размера N см. на рисунке 2.

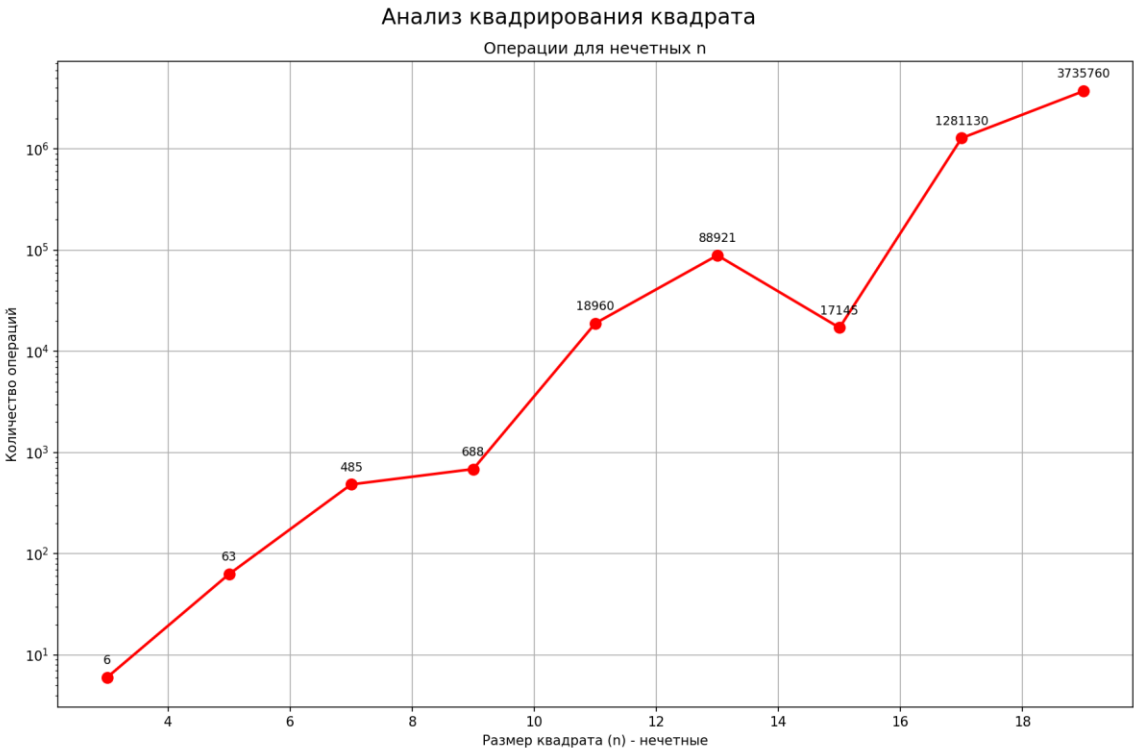

Рисунок 2. Зависимость кол-ва операции от размере N

Описание функций и структур данных.

Для хранения координат используемых квадратов использовалась структура SquareDate хранившая координаты по x и y, а также размер стороны квадрата.

Реализован класс Square который выполнял кадрирование и хранил необходимую информацию, с конструктором принимающим числовое значение N и начинавшим рекурсивное кадрирование квадрата. Класс хранил следущие поля:

int n – размер квадрата

int cnt_squares – кол-во используемых квадратов меньших N-1

std::vector<SquareDate> squares – данных о текущей вариации расположения квадратов меньших N-1.

int min_cnt_squares – минимальное кол-во квадратов в лучшем решении

std::vector<SquareDate> best_squares – данные о лучшей конфигурации квадрата.

int area – площадь квадрата.

long long cnt_ops – счетчик кол-во операций.

Основные методы класса:

bool can_place(int x, int y, int size) – метод определял возможность расположения нового квадрата по координатам и размеру стороны в текущей конфигурации.

void place_square(int x, int y, int size, int value) – метод устанавливал или удалял квадрат из текущей конфигурации.

void find_first_coord(int start_y, int& new_x, int& new_y) – метод искал первую свободную ячейку для попытку установки квадрата.

void backtrace(*int y*, *int depth*) – основной метод класса реализующий поиск конфигурации с помощью рекурсии. Метод ничего не возвращает, только сохраняет информацию в полях класса и ей же оперирует. Метод принимает числовое значение y – это значение откуда по оси y искать свободную клетку для установки квадрата, чтобы каждый раз не начинать с начала сетки. Также метод принимает счетчик глубины рекурсии, чтобы выводить debug информацию. Описание работы метода:

В самом начале проверяется необходимость продолжение работы алгоритма в данной ветке путем сравнения кол-ва используемых квадратов и уже найденного минимального (лучшего) решения. Далее в случае, если найденная конфигурация лучше прежней, то происходит замена данных. Затем поиск следующих свободных координат и попытка установки нового квадрата туда с использованием цикла размера квадарата, после идет вызов того же метода backtrace и после конечной точки рекурсии идет возврат из стека вызова и удаление последнего варианта квадрата.

Код выполнения работы см. в Приложении А.

Приложения

Приложение А

main.cpp:

```
#include <iostream>
#include <vector>
#include <chrono>
#include <fstream>

using namespace std::chrono;

#define DEBUG false
#define COLLECT_DATA true
#define INDEX(x, y, n) ((y) * (n) + (x))

struct SquareDate{
    int x, y, size;
};

class Square{
private:
    int n;
    int cnt_squares{0};
    std::vector<int> field;
    std::vector<SquareDate> squares;

    int min_cnt_squares;
    std::vector<SquareDate> best_squares;

    int area;

    long long cnt_ops{0};

    std::string indent(int depth){
        return std::string(depth * 2, ' ');
    }

    bool can_place(int x, int y, int size){
        if (x + size > n || y + size > n) return false;

        for(int yi = y; yi < y + size; yi++){
            for(int xi = x; xi < x + size; xi++){
                if(field[INDEX(xi, yi, n)]) return false;
            }
        }
        return true;
    }

    void place_square(int x, int y, int size, int value){
        int delta = size * size;

        for (int yi = y; yi < y + size; yi++){
            for (int xi = x; xi < x + size; xi++){
                field[INDEX(xi, yi, n)] = value;
```

```
            }
        }

        if (value){
            cnt_squares++;
            area -= delta;
        } else {
            cnt_squares--;
            area += delta;
        }
    }

    void find_first_coord(int start_y, int& new_x, int& new_y){
        for (int yi = start_y; yi < n; yi++){
            for (int xi = 0; xi < n; xi++){
                if (!field[INDEX(xi, yi, n)]){
                    new_x = xi;
                    new_y = yi;
                    return;
                }
            }
        }
        new_x = new_y = -1;
    }

    void backtrace(int y, int depth){
        cnt_ops++;
        if (cnt_squares >= min_cnt_squares - 1 && area > 0){
            if (DEBUG) std::cout << indent(depth)
                << "- Текущее решение хуже лучшего, возвращяюсь на
этап назад..." << std::endl;
            return;
        }
        if (area == 0) {
            if (cnt_squares < min_cnt_squares){
                min_cnt_squares = cnt_squares;
                best_squares = squares;
                if (DEBUG) std::cout << "\n" << indent(depth) << ">>>
Найдено решение: "
                    << cnt_squares << " квадратов" << " <<<\n" <<
std::endl;
            }
            return;
        };

        int next_x, next_y;
        find_first_coord(y, next_x, next_y);
        if (next_x == -1) return;

        int max_s = std::min(n - next_x, n - next_y);

        for (int size = max_s; size >= 1; size--){
            if (can_place(next_x, next_y, size)){
                if (DEBUG) std::cout << indent(depth) << "+ Пробуем "
<< size << "x" << size
                    << " в точку (" << next_x << "," << next_y <<
")\n";
```

```
                place_square(next_x, next_y, size, 1);
                squares.push_back(SquareDate{next_x + 1, next_y + 1,
size});

                backtrace(next_y, depth + 1);

                place_square(next_x, next_y, size, 0);
                squares.pop_back();
            }
        }
    }

public:
    Square(int n) : n(n) {
        if (n % 2 == 0){
            cnt_ops++;
            int size = n / 2;
            best_squares.push_back(SquareDate({1, 1, size}));
            best_squares.push_back(SquareDate({1 + size, 1, size}));
            best_squares.push_back(SquareDate({1, 1 + size, size}));
            best_squares.push_back(SquareDate({1 + size, 1 + size,
size}));
            min_cnt_squares = 4;
            return;
        }

        field.assign(n * n, 0);
        min_cnt_squares = n * n + 1;
        area = n * n;

        for(int size = n - 1; size >= (n + 1) / 2; size--){
            place_square(0, 0, size, 1);
            squares.push_back({1, 1, size});

            backtrace(0, 1);

            place_square(0, 0, size, 0);
            squares.pop_back();
        }
    };

    int get_cnt_squares() {return min_cnt_squares;};
    std::vector<SquareDate> get_squares() {return best_squares;};
    int get_cnt_ops() {return cnt_ops;};

    void draw_square(){
        std::vector<std::vector<int>> matrix;
        matrix.resize(n);
        for (int y = 0; y < n; y++){
            matrix[y].assign(n, 0);
        }

        for (int i = 0; i < best_squares.size(); i++){
            int x = best_squares[i].x - 1;
            int y = best_squares[i].y - 1;
            int size = best_squares[i].size;
```

```
            for (int yi = y; yi < y + size; yi++){
                for (int xi = x; xi < x + size; xi++){
                    matrix[yi][xi] = i;
                }
            }
        }

        for (int y = 0; y < n; y++){
            for (int x = 0; x < n; x++){
                std::cout << matrix[y][x] << " ";
            }
            std::cout << std::endl;
        }
    }
};

void collect_data(int n){
    std::ofstream file("data.csv");

    if (file.is_open()){
        file << "n,cnt_ops,time" << std::endl;

        for(int ni = 2; ni < n; ni++){
            auto start = high_resolution_clock::now();
            Square s(ni);
            auto stop = high_resolution_clock::now();

            auto duration = duration_cast<microseconds>(stop - start);
            file << ni << "," << s.get_cnt_ops() << "," <<
duration.count() / 1000000 << '.' << duration.count() % 1000000 <<
std::endl;

        }
    file.close();
    }
}

int main(){
    int n;

    if (DEBUG) std::cout << "Введите N (размер квадрата): ";
    std::cin >> n;

    if (n < 2){
        return 1;
    }

    auto start = high_resolution_clock::now();
    Square s(n);
    auto stop = high_resolution_clock::now();

    auto duration = duration_cast<microseconds>(stop - start);
    std::cout << duration.count() / 1000000 << '.' << duration.count()
% 1000000 << std::endl;

    if (DEBUG) std::cout << "Кол-во операций вставки квадратов: " <<
s.get_cnt_ops() << std::endl;
```

```
    if (DEBUG) std::cout << "Минимальное кол-во квдаратов из которых
можно собрать большой квадрат: ";
    std::cout << s.get_cnt_squares() << std::endl;

    if (DEBUG) std::cout << "X | Y | SIZE" << std::endl;
    auto res = s.get_squares();
    for (auto& sq : res){
        std::cout << sq.x << " " << sq.y << " " << sq.size <<
std::endl;
    }

    if (DEBUG) std::cout << "\nВизуализация: " << std::endl;
    if (DEBUG) s.draw_square();

    if (COLLECT_DATA) collect_data(21);

    return 0;
}
```

Приложение В
Тестирование

N = 2
4
1 1 1
2 1 1
1 2 1
2 2 1

N = 3
6
1 1 2
3 1 1
3 2 1
1 3 1
2 3 1
3 3 1

N = 4
4
1 1 2
3 1 2
1 3 2
3 3 2

N = 5
8
1 1 3
4 1 2
4 3 2
1 4 2
3 4 1
3 5 1
4 5 1
5 5 1

N = 6
4
1 1 3
4 1 3
1 4 3
4 4 3

N = 7
9
1 1 4
5 1 3
5 4 2
7 4 1
1 5 3
4 5 1
7 5 1
4 6 2
6 6 2

N = 8
4
1 1 4
5 1 4
1 5 4
5 5 4

N = 9

6
1 1 6
7 1 3
7 4 3
1 7 3
4 7 3
7 7 3

N = 10
4
1 1 5
6 1 5
1 6 5
6 6 5

N = 11
11
1 1 7
8 1 4
8 5 4
1 8 4
5 8 2
7 8 1
7 9 3
10 9 2
5 10 2
10 11 1
11 11 1

N = 12
4
1 1 6
7 1 6
1 7 6
7 7 6

N = 13
11
1 1 7

8 1 6
8 7 2
10 7 4
1 8 6
7 8 1
7 9 3
10 11 1
11 11 3
7 12 2
9 12 2

N = 14
4
1 1 7
8 1 7
1 8 7
8 8 7

N = 15
6
1 1 10
11 1 5
11 6 5
1 11 5
6 11 5
11 11 5

N = 16
4
1 1 8
9 1 8
1 9 8
9 9 8

N = 17
12
1 1 9
10 1 8
10 9 2

12 9 4
16 9 2
1 10 8
9 10 1
9 11 3
16 11 2
12 13 1
13 13 5
9 14 4

N = 18
4
1 1 9
10 1 9
1 10 9
10 10 9

N = 19
13
1 1 13
14 1 6
14 7 6
14 13 2
16 13 4
1 14 6
7 14 6
13 14 1
13 15 3
16 17 1
17 17 3
13 18 2
15 18 2

N = 20
4
1 1 10
11 1 10
1 11 10
11 11 10